

Question 1:

Dans la RFC 7231, nous trouvons que certaines méthodes HTTP sont considérées comme sûres (safe) ou idempotentes, en fonction de leur capacité à modifier (ou non) l'état de l'application. Lisez les sections 4.2.1 et 4.2.2 de la RFC 7231 et répondez : parmi les méthodes mentionnées dans l'activité 2, lesquelles sont sûres, non sûres, idempotentes et/ou non idempotentes?

Parmi les méthodes de l'activité 2 (smoke test), seulement la requête de vérification du stock:

`client.get(f'/stocks/{product_id}')` est sûre, car c'est essentiellement la seule requête qui est read-only et qui ne modifie aucun état sur le serveur.

Concernant l'idempotence, la requête de suppression d'une commande

`client.delete(f'/orders/{order_id}')` et de vérification du stock `client.get(f'/stocks/{product_id}')` sont idempotentes. Dans le cas de la suppression, envoyer la requête donne le même résultat: la commande n'existe plus. Pour la requête de vérification, on obtient la même information, donc c'est idempotent.

Les requêtes POST ajoutant des produits, stocks, ou commandes ne sont ni sûres ni idempotentes, étant donnée qu'ils modifient l'état du serveur (non sûre). De plus, effectuer un POST ne sera pas idempotent, car on se retrouve avec deux produits (par exemple) avec des id distincts.

Question 2:

Décrivez l'utilisation de la méthode join dans ce cas. Utilisez les méthodes telles que décrites à Simple Relationship Joins et Joins to a Target with an ON Clause dans la documentation SQLAlchemy pour ajouter les colonnes demandées dans cette activité. Veuillez inclure le code pour illustrer votre réponse.

Mon approche dans ce labo a été d'utiliser le "Join to a target with an ON clause". Il s'agit de spécifier la table ciblée qu'on veut joindre, et la condition de jointure (sur quoi joindre):

```
results = session.query(
    Stock.product_id,
    Stock.quantity,
    Product.name,
    Product.sku,
    Product.price
).join(Product, Stock.product_id == Product.id).all()
```

Ici, concrètement, on passe deux argument: Product (qui est la table cible à joindre), et la condition de jointure, qui signifie que les champs product_id de la stable Stock seront comparés ave le champ de id de la table Product afin d'effectuer la jointure.

Ce code python est l'équivalent de ce code SQL:

```
SELECT stocks.product_id, stocks.quantity, products.name, products.sku, products.price
FROM STOCKS
JOIN products ON stocks.product_id = procuts.id
```

Question 3:

Quels résultats avez-vous obtenus en utilisant l'endpoint POST /stocks/graphql-query avec la requête suggérée ? Veuillez joindre la sortie de votre requête dans Postman afin d'illustrer votre réponse.

À la première exécution, le résultat me retourne des null, étant donné que Redis n'était pas peuplé initialement:

```
{
  "data": {
    "product": null
  },
  "errors": null
}
```

Après avoir effectué un POST /stocks dans Postman, et réessayé le endpoint GraphQL, on peut apercevoir le produit ajouté:

```
{
  "data": {
    "product": {
      "id": 1,
      "name": "",
      "quantity": 56
    }
  },
  "errors": null
}
```

Question 4:

Quelles lignes avez-vous changé dans update_stock_redis? Veuillez joindre du code afin d'illustrer votre réponse.

Avant, update_stock_redis n'enregistrait que la quantité dans redis. Puisque GraphQL ne lit que Redis, il faut récupérer les autres infos depuis MySQL:

```
session = get_sqlalchemy_session()
product = session.query(Product).filter_by(id=product_id).first()
pipeline.hset(f"stock:{product_id}", mapping = {
    "quantity": new_quantity,
    "name": product.name,
    "sku": product.sku,
    "price": float(product.price)
})
```

Question 5:

Quels résultats avez-vous obtenus en utilisant l'endpoint POST /stocks/graphql-query avec les améliorations ? Veuillez joindre la sortie de votre requête dans Postman afin d'illustrer votre réponse.

On voit maintenant les informations additionnelles, comme le nom du produit, le prix, et le sku:

```
{
  "data": {
    "product": {
      "id": 1,
      "name": "Laptop ABC",
      "price": 1999.99,
      "quantity": 56,
      "sku": "LP12567"
    }
  },
  "errors": null
}
```

Question 6:

Examinez attentivement le fichier docker-compose.yml du répertoire scripts, ainsi que celui situé à la racine du projet. Qu'ont-ils en commun ? Par quel mécanisme ces conteneurs peuvent-ils communiquer entre eux ? Veuillez joindre du code YML afin d'illustrer votre réponse.

Les 2 fichiers docker-compose possèdent chacun une déclaration du réseau labo03-network comme externe. Cela veut dire qu'ils utilisent un réseau externe déjà existant. De plus, ce qui permet de communiquer entre eux est le paramètre `driver: bridge`. Dans ce cas, Docker fournit un DNS interne qui permet à chaque conteneur d'être adressé par son nom de service.

```
# docker-compose.yml (racine)
networks:
  labo03-network:
    driver: bridge
    external: true

# scripts/docker-compose.yml
networks:
  labo03-network:
    driver: bridge
    external: true
```

Pipeline CI/CD:

Ma pipeline est constituée de 2 jobs:

La première consiste à exécuter les tests unitaires. Pour s'y faire, on installe les dépendances, setup le fichier .env, on démarre le conteneur, on vérifie que MySQL et Redis fonctionnent bien, et finalement on exécute les tests. Prendre note que ces tests ne s'effectuent pas sur la VM, mais plutôt sur les serveurs de GitHub

La deuxième job consiste à effectuer le déploiement sur la VM. Ce job s'effectue seulement si la première job (les tests) a passé. Il s'agit surtout de copier/cloner les fichiers du repo localement sur la VM, créer le fichier .env, créer le réseau docker, build & docker compose up, et finalement, si MySQL & Redis fonctionnent bien et qu'il n'y a eu aucune erreur jusqu'à présent, on peut considérer l'application déployée!

Voici des images démontrant le résultat de la pipeline fonctionnelle:

ci.yml

on: push



Run tests

```
1 ▶ Run docker compose exec -T store_manager pytest
11 ===== test session starts =====
12 platform linux -- Python 3.11.15, pytest-9.0.3, pluggy-1.6.0
13 rootdir: /app
14 configfile: pyproject.toml
15 collected 2 items
16
17 tests/test_store_manager.py .. [100%]
18
19 ===== 2 passed in 0.67s =====
```

Déploiement automatique

succeeded 1 hour ago in 1m 10s

Search logs

↺ ⚙

- > ✓ Set up job 2s
- > ✓ Cleanup conteneur 43s
- > ✓ Checkout 1s
- > ✓ Copier vers le répertoire 0s
- > ✓ Créer le fichier .env 0s
- > ✓ Créer le réseau Docker 1s
- > ✓ Build et démarrer l'application 8s
- ✓ Vérifier le déploiement 11s

```
1 ▶ Run cd ~/log430-labo3-E26
31 mysql is alive
32 Deployment succeeded!
33 CONTAINER ID IMAGE COMMAND NAMES CREATED STATUS
34 3cfb5a11d2f6 log430-labo3-e26-store_manager "python store_manage... 17 seconds ago Up 10 seconds
0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp log430-labo3-e26-store_manager-1
35 aaea16f6e9bb redis:7 "docker-entrypoint.s..." 17 seconds ago Up 17 seconds (healthy)
0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp log430-labo3-e26-redis-1
36 719510efc8bc mysql:8.4.7 "docker-entrypoint.s..." 17 seconds ago Up 17 seconds (healthy)
0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp log430-labo3-e26-mysql-1
37 adbe9ba01e7a mongo:6 "docker-entrypoint.s..." 11 days ago Up About an hour
0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp mongo
```